Supplementary Document

Use LLMs to Build Repair Tools, Not Patch Generation: Cost-Effective Automated Program Repair

Anonymous submission

This document carries what the main paper's page budget could not. Nothing here introduces a claim the paper does not already make: each item is the tabular or graphical form of a number the Results section gives, or a measurement the paper cites without room to print. Assignment ids (R1–R5, C1–C6), the arm names, and every metric definition are the main paper's. Values are generated from `results/` by the released scripts (`run.py tables`, `run.py figures`); none is transcribed.

Where the Repairs Come From

Table S1 decomposes every repair by the strategy that produced it, with the mean patch each leaves and whether it carries a misconception label, pooled over both builders. Shares are over repaired submissions; the LLM row belongs to CoEP_{LF} and sits outside that denominator. The last rows are the framework-only ablation, the identical tool with the builder's rules removed, which is the same artifact under either builder because nothing the model wrote survives in it.

Repair source	share (%)	mean RPS	labelled
1. rules	24.6	0.100	•
2. reference transplant	75.4	0.581	–
3. LLM (CoEP_{LF})	8.2	0.724	–
Framework-only ablation (builder rules removed), first column is RR (%) rather than share			
ablated	98.69	0.492	–
full tool	99.06	0.463	–

Table S1: Repair source, patch size, and labelling, with the framework-only ablation.

What Escalation Costs, Assignment by Assignment

The main paper reports the escalation rate e as a corpus figure. It is not uniform: Table S2 gives it per assignment alongside the calls it implies, the local CPU each submission spends, and the one-off build wall clock. Two readings matter for a department sizing this. The assignments that escalate most are the ones whose local patches are largest rather than the hardest ones, and the compute is not free: 5,940 submissions cost 4.8 core hours of local search, on top of roughly 22 minutes of build per assignment per builder. Neither figure appears in the dollar totals, which price model tokens alone.

	e (%)	calls	ATT_{cpu} (s)	build (min)
R1	0.7	8	2.28	32
R2	22.5	196	3.18	27
R3	0.0	0	1.83	18
R4	9.2	66	3.43	28
R5	0.9	2	4.79	18
C1	13.2	116	3.17	20
C2	23.3	261	2.22	38
C3	13.0	18	6.94	3
C4	5.3	4	1.34	12
C5	15.6	24	4.86	4
C6	66.7	4	6.58	6
All	**11.8**	**699**	**2.88**	**206**

Table S2: Escalation rate, calls, local CPU per submission, and build wall clock per assignment, over both builders.

How the Arms Fare as the Pool Thins

The main paper's ablation compares the tool with and without the builder's rules on the benchmark as it stands, where every assignment arrives with hundreds of correct submissions already in hand. Table S3 varies that: each tool is held fixed and only the number of correct programs the transplant may draw on is restricted. `run.py pool` reproduces it.

Two properties of the design matter for reading the table. The 146 submissions are drawn only from those no builder was shown; roughly half of each assignment's buggy pool ends up in the build set, and scoring those would ask whether a rule remembers the program it was written from. And each sampled pool is drawn three times, since one draw of k correct programs is one accident; the brackets are the range across draws.

Coverage separates the layers only where the pool is gone. At one, two or five correct programs the arms are within half a point of each other and inside the spread across draws, which at this sample size is no separation at all; the full campaign, two orders of magnitude larger, does separate them, at 99.06% against 98.69%. With no pool the substrate has

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

nothing to transplant and repairs 8.9% where the same tool with its rules repairs 29.5%.

Patch size separates them at every pool size that has one, and by more as the pool thins: 4.1% apart on the whole pool, 8.9% at five, 12.4% at one. The k=0 row is not comparable in that column, since the two arms repair different and differently hard sets.

This is a repair-time restriction, not a cold start. Each tool was built with the whole pool available, since its worked pairs are drawn from it, so what the table isolates is the substrate's dependence on the pool at deployment. A true cold start would rebuild at each pool size and would cost a model call per size.

	RR (%)		RPS	
pool	CoEP_{fw}	full	CoEP_{fw}	full
0	8.9	29.5	0.111	0.203
1	97.5 [96.6, 98.6]	97.9 [97.3, 98.6]	1.109 [0.974, 1.351]	0.971 [0.867, 1.162]
2	99.1 [98.6, 100.0]	99.3 [98.6, 100.0]	0.884 [0.794, 1.022]	0.792 [0.715, 0.906]
5	99.5 [98.6, 100.0]	99.8 [99.3, 100.0]	0.663 [0.619, 0.729]	0.604 [0.573, 0.665]
all	100.0	100.0	0.490	0.470

Table S3: Repair rate and patch size as the reference pool is restricted to k correct programs, over 146 held-out submissions; brackets are the range over three draws of each pool.

Robustness

Table S4 puts three axes side by side: assignment difficulty, the choice of builder, and the escalation threshold. The difficulty bands group the eleven assignments as CS1, Codeforces 800–1000, and Codeforces 1200–2000, the last folding C6's three submissions in rather than reporting them alone.

Stratum	Refactory	PyDex RR (%) / RPS	CoEP_{LF}
Difficulty (`claude-sonnet-5`)			
CS1	90.8 / 0.397	98.6 / 0.382	99.4 / 0.279
CF 800–1000	83.1 / 0.663	99.4 / 0.942	100.0 / 0.572
CF 1200–2000	54.5 / 0.815	100.0 / 0.800	100.0 / 0.693
Builder			
`claude-sonnet-5`	–	99.0 / 0.598	99.7 / 0.404
`gpt-5.4-mini`	–	98.9 / 0.669	99.7 / 0.417
Threshold, 29 measured settings			
range	–	–	99.7 / 0.378–0.451
spend		$12.47–$15.61 total	

Table S4: Difficulty, builder, and escalation threshold.

Figure S1 plots every measured threshold setting against the baselines it is compared to. The point is the emptiness between the cluster and the baselines: the whole family spans $12.47–$15.61 of total spend, so which setting ships cannot change a conclusion. The shipped rule, escalate a passing local patch whose RPS exceeds 1, is marked with a star and is the canonical run rather than the sweep's reconstruction of it.

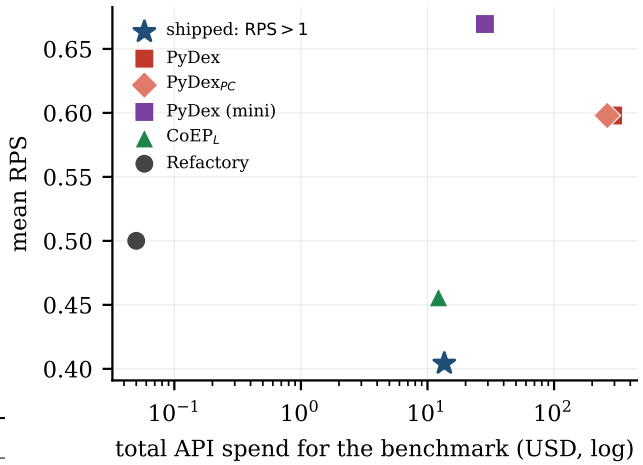


Figure S1: Mean RPS against total spend. Each small marker is one threshold setting; the star is the shipped rule. Repair rate is identical across all of them.

Rule Provenance

Table S5 lists the rules that carry the most submissions, counted as distinct submissions rather than as applications, since a repair may compose several rules. The distinction that matters is reuse: of the 128 rules the builders wrote and that fired, exactly one fired on more than one assignment, `add_missing_call_parens`, which describes a Python trap rather than anything about a problem. The always-on rules recur everywhere by construction, which is why the builder's layer is priced per assignment and they are not.

Rule	where	submissions
`generic_flip_compare`	always-on	263
`append_missing_return_len_after_loop`	R1	114
`slice_case_check_to_equality`	C2	108
`generic_shift_index`	always-on	74
`strict_one_should_be_exactly_one`	R2	70
`count_compare_relax`	R2	58
`generic_shift_range`	always-on	50
`capitalize_title_to_swapcase`	C2	43
128 builder-written rules fired, 1 on more than one assignment		

Table S5: The eight rules repairing the most submissions.

The eight rules self-learning absorbed read as misconceptions in the same way: `counter_var_confusion`, `uniqueness_count_gt1_to_neq1`, `index_vs_count_and_uncalled_bool` and `missing_else_false` on R2 with `claude-sonnet-5`; `count_equality_for_uniqueness`, `count_uniqueness_instead_of_direct_membership` and `missing_call_and_predicate_fix` on R2 with `gpt-5.4-mini`; and `binarysearch_zero_exit` on C3.

Reproducibility

The repository contains the harness, every prompt, the 22 synthesized repairers with their build costs, and the per-submission event log each arm wrote. Four commands regenerate every number in both documents from those logs without touching a model: `run.py tables`, `run.py figures`, `run.py numbers`, which recomputes each quantity and reports whether the string still occurs in the paper, and `run.py stats`. The measured threshold settings are archived as `results/threshold_sweep.json`.

Two dependencies are outside our control and worth stating. Both builders are closed commodity models reached through their providers' APIs, so a rebuild cannot be guaranteed to reproduce the same rules; what is released is the tools those builds produced, which are ordinary Python and run offline. Refactory's replication package is its authors' and is cloned rather than vendored.